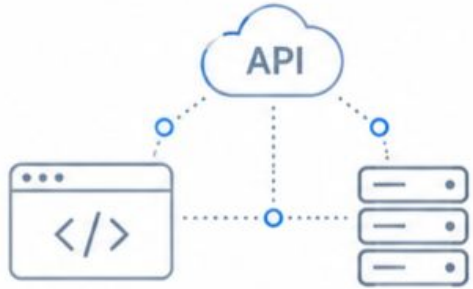
REST API и RESTful API

Ключевые понятия и их различия



REST

REST — архитектурный стиль для построения распределённых систем.

REST API

REST API — API, построенное с использованием идей REST.

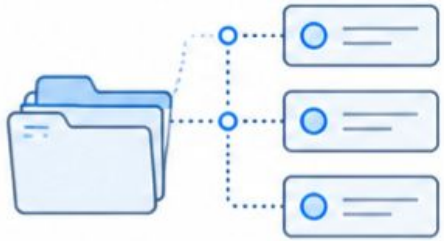
RESTful API

RESTful API — API, которое хорошо следует принципам REST.

Главная цель лекции: понять разницу между REST API и RESTful API.

Главная идея REST — ресурсы

Клиент работает с ресурсами, а не с функциями сервера.



Плохо

URI описывают действия (функции сервера):

```
/getStudent
/createStudent
/deleteStudent
```

Лучше

URI описывают ресурсы:

```
/students
/students/42
```

Ресурс отвечает на вопрос: «С чем мы работаем?»

HTTP-метод определяет действие

URI → ресурс, **HTTP method** → действие



GET	/students
POST	/students
GET	/students/42
PATCH	/students/42
DELETE	/students/42

Не нужно писать:

`/deleteStudent`

Действие уже задаётся HTTP-методом, а не названием URL.

Действие задаётся методом, а не названием URL.

REST использует возможности HTTP



HTTP methods

Методы HTTP определяют действие над ресурсом.

GET	→	получить
POST	→	создать
PUT	→	заменить
PATCH	→	изменить
DELETE	→	удалить



Status codes

Коды состояния HTTP сообщают результат выполнения запроса.

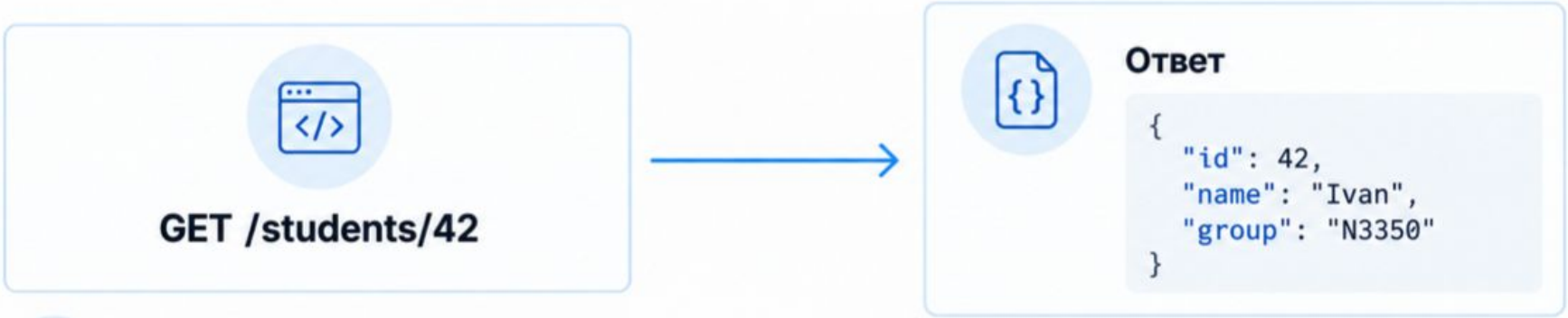
200	OK
201	Created
204	No Content
400	Bad Request
404	Not Found
500	Internal Server Error



RESTful API использует стандартную семантику HTTP.

Представление ресурса

REST = Representational State Transfer



Клиент получает не сам объект на сервере, а его представление.

Обычно представление передаётся в **JSON**.

Stateless

Каждый запрос должен быть самодостаточным.



Сервер не должен зависеть от того, какие запросы клиент отправлял до этого.

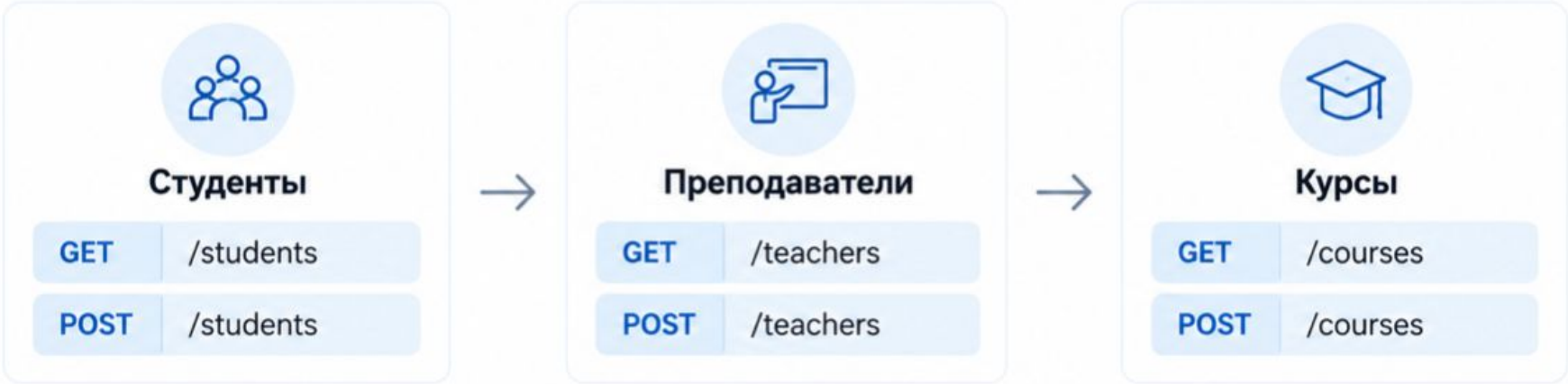




Stateless не означает, что сервер не хранит данные. Он не должен хранить необходимый контекст предыдущего взаимодействия клиента.

Uniform Interface

REST API должен быть единообразным и предсказуемым.



Одинаковые правила делают API проще для понимания.

Что делает API RESTful

Ключевые принципы REST и их роль



Client–Server

Разделение клиента и сервера, независимая эволюция сторон.



Stateless

Каждый запрос содержит всю необходимую информацию, сервер не хранит состояние клиента.



Cacheable

Ответы могут кэшироваться для повышения производительности и снижения нагрузки.



Uniform Interface

Единый и последовательный интерфейс взаимодействия (ресурсы, методы, статусы и т. д.).



Layered System

Клиент не знает, напрямую он работает с конечным сервером или через промежуточные слои (прокси, шлюзы).



Code on Demand — необязательный

Сервер может передавать исполняемый код клиенту (например, скрипты), но этот принцип используется редко.



RESTful — это не просто красивые URL. Это соблюдение архитектурных принципов REST.

REST API vs RESTful API

Сравнение на примере удаления студента

Хуже

Просто HTTP API

```
POST /api/deleteStudent
```

Работает, но плохо соответствует REST.

Лучше

Ближе к RESTful

```
DELETE /students/42
```

Ресурс + метод = понятная семантика.

Но даже хороший URL сам по себе ещё не гарантирует, что вся система RESTful.

Что студент должен запомнить

1. **REST** — архитектурный стиль.
2. **Ресурс** — центральное понятие REST.
3. **URI** определяет ресурс.
4. **HTTP-метод** определяет действие.
5. **RESTful API** использует семантику HTTP.
6. **RESTful API** должен быть stateless и единообразным.
7. **RESTful** ≠ просто GET /users.



РЕЗУЛЬТАТ

REST API — это способ организовать взаимодействие с ресурсами, а **RESTful API** — насколько хорошо это взаимодействие следует принципам REST.

Как правильно писать REST API

Примеры правильного проектирования endpoint в разных случаях

	Ситуация	Правильно		Неудачный вариант	
1	Получить всех студентов	GET	/students	GET	/getStudents
2	Получить одного студента	GET	/students/42	GET	/getStudent?id=42
3	Создать студента	POST	/students	POST	/createStudent
4	Полностью заменить студента	PUT	/students/42	POST	/updateStudent
5	Частично изменить студента	PATCH	/students/42	POST	/changeStudent
6	Удалить студента	DELETE	/students/42	POST	/deleteStudent
7	Получить студентов группы	GET	/students?group=N3350	GET	/getStudentsByGroup/N3350
8	Найти по имени	GET	/students?name=Ivan	POST	/findStudent
9	Сортировка	GET	/students?sort=name	GET	/sortStudentsByName
10	Пагинация	GET	/students?page=2&limit=20	GET	/getSecondPage



Главная формула: HTTP METHOD + RESOURCE + PARAMETERS

Коллекция и конкретный ресурс

Два уровня работы с ресурсами в REST API



Коллекция

`/students`

Работа со всеми студентами

GET	/students
POST	/students



Конкретный ресурс

`/students/42`

Работа с одним студентом

GET	/students/42
PUT	/students/42
PATCH	/students/42
DELETE	/students/42



/students → коллекция | **/students/42** → конкретный объект

Связанные ресурсы

Как показывать связи между сущностями в URI



GET /students/42/courses

Получить курсы студента

POST /students/42/courses

Добавить курс студенту

GET /students/42/courses/7

Получить конкретный курс

URI показывает связь: student 42 → courses → course 7.

Фильтрация, сортировка и пагинация

Для выборки коллекции используем query parameters

Фильтрация
Выборка по условию.

`GET /students?group=N3350`

Несколько фильтров
Комбинируем параметры.

`GET /students?group=N3350&status=active`

Сортировка
Определяем порядок.

`GET /students?sort=name`

Пагинация
Разбиваем на страницы.

`GET /students?page=2&limit=20`

Комбинация
Используем вместе.

`GET /students?group=N3350&sort=name&page=2&limit=20`



Не нужно
Использовать действия в пути URL.

`/students/getByGroup/N3350`

`/students/getPage/2`

Query parameters уточняют выборку, а не создают новый ресурс.

PUT или PATCH?

Как выбрать правильный метод обновления ресурса

PUT

Полностью заменить ресурс

PUT /students/42

```
{
  "name": "Ivan",
  "group": "N3351",
  "email": "ivan@example.com"
}
```

PATCH

Изменить только часть ресурса

PATCH /students/42

```
{
  "group": "N3351"
}
```

PUT → заменить весь ресурс | **PATCH** → изменить часть ресурса

А если действие не CRUD?

Как проектировать специальные действия в REST API



Часто встречается

```
POST  /orders/42/cancel
```

Понятно, но ближе к RPC-style action.

Более ресурсный вариант

```
POST  /orders/42/cancellations
```

Создаём подчинённый ресурс, который представляет отмену заказа.

Или изменение состояния

```
PATCH  /orders/42
```

```
{
  "status": "cancelled"
}
```

Обновляем ресурс, изменяя его состояние.

Не каждое действие идеально укладывается в CRUD. Выбор зависит от предметной области.

Как думать при проектировании endpoint

Простой алгоритм для выбора правильного REST URI

1



С каким ресурсом я работаю?

Определите, какую сущность представляет ресурс.

Пример:

`Student`

REST URI:

`/students`

2



С одним или с коллекцией?

Решите, работаете ли вы со всей коллекцией или с конкретным объектом.

Коллекция (все объекты):

`/students`

Конкретный объект:

`/students/42`

3



Что я хочу сделать?

Выберите HTTP-метод в зависимости от желаемого действия.

получить	GET
создать	POST
заменить	PUT
изменить	PATCH
удалить	DELETE

Получаем набор endpoint

Итоговые REST URI для ресурса Student:

GET	`/students`
POST	`/students`
GET	`/students/42`
PATCH	`/students/42`
DELETE	`/students/42`



Главная идея: Сначала определяем ресурс, потом уровень (коллекция или объект), затем выбираем HTTP-метод.